

รายงานโครงงานวิชา Machine Learning

การทำนายจำนวนการเช่าจักรยานรายชั่วโมง ในกรุงโซลจากสภาพอากาศและปฏิทิน

Seoul Bike Sharing Demand — Regression

รหัสனிสิต	ชื่อ-นามสกุล
(กรอกรหัสนิสิต)	(กรอกชื่อ-นามสกุล)
(กรอกรหัสนิสิต)	(กรอกชื่อ-นามสกุล)
(กรอกรหัสนิสิต)	(กรอกชื่อ-นามสกุล)

ตุลาคม 2569

github.com/Sh080205/seoul-bike-demand

สารบัญ

บทคัดย่อ

1. นิยามปัญหา
2. ข้อมูล
3. การแบ่งข้อมูลและการป้องกัน leakage
4. การสำรวจข้อมูล (EDA)
5. Feature engineering และ preprocessing
6. Feature selection
7. PCA
8. Baseline และการเทียบโมเดล
9. Hyperparameter tuning
10. การเลือกโมเดลและทรัพยากรที่ใช้
11. ผลบนชุดทดสอบและ error analysis
12. การทำนายข้อมูลใหม่
13. ความท้าทายที่พบ
14. ข้อจำกัดและแนวทางพัฒนาต่อ
15. สรุป

ภาคผนวก ก. เช็คลิสต์ป้องกัน leakage · ข. การรันซ้ำ

เอกสารอ้างอิง

บทคัดย่อ

โครงการนี้สร้างโมเดล regression ทำนาย **จำนวนจักรยานที่ถูกเช่าในแต่ละชั่วโมง** ของระบบ Seoul Bike ทั้งเมือง จากข้อมูลที่เราสามารถหาได้ คือพยากรณ์อากาศและปฏิทิน โดยใช้ชุดข้อมูล Seoul Bike Sharing Demand จาก UCI (8,760 ชั่วโมง ค.ศ. 2017 – พ.ย. 2018) ข้อมูลถูกแบ่ง train/test **ตามวัน** เพื่อไม่ให้ชั่วโมงของวันเดียวกันรั่วข้ามสองฝั่ง และทุกการตัดสินใจใช้ 5-fold GroupKFold บน train เท่านั้น เราเทียบ 4 ตระกูลโมเดล (Linear Regression, kNN, Random Forest, HistGradientBoosting) กับ baseline 3 แบบ พร้อมทำ feature selection 3 แนวทางและ PCA โมเดลที่เลือกคือ **HistGradientBoostingRegressor** ได้ CV MAE 109.0 ± 8.1 คัน/ชม. และเมื่อประเมินชุดทดสอบ ครั้งเดียวได้ MAE 97.4 คัน/ชม. และ R^2 0.919 จุดอ่อนหลักคือชั่วโมงที่ฝนตกและเหตุการณ์พิเศษที่ไม่มีใน feature เช่น พายุไต้ฝุ่นและเทศกาล

109.0

CV MAE (คัน/ชม.)
5-fold GroupKFold

97.4

test MAE (คัน/ชม.)
ประเมินครั้งเดียว

0.919

test R^2

-73%

MAE เทียบ baseline
ที่ใช้แค่เวลา (401.0)

1. นิยามปัญหา

หัวข้อ	รายละเอียด
ชนิดงาน	Regression
ทำนายอะไร	จำนวนจักรยานที่ถูกเช่าในชั่วโมงนั้น (rented_bike_count , คัน/ชั่วโมง) ของทั้งระบบ
ใช้ทำอะไร	ให้ผู้ให้บริการวางแผนจำนวนจักรยานที่ต้องพร้อมใช้ และจัดคนกระจายจักรยานส่งคืนในชั่วโมงที่คาดว่าจะมีคนเช่ามาก
input	สภาพอากาศ (อุณหภูมิ, ความชื้น, ลม, ทิศนวิสัย, dew point, แสงแดด, ฝน, หิมะ) + เวลา (ชั่วโมง, วันในสัปดาห์, เดือน) + ฤดู + วันหยุด
เมตริกหลัก	MAE (คัน/ชม.) — ดีความตรงว่า "หายคลาดเคลื่อนกี่คัน" และไม่ถูกชั่วโมงฟัดไม่กี่ชั่วโมงครอบงำเหมือน RMSE (target เบ้า: mean 756.8 > median 582.5)
เมตริกรอง	RMSE (ดูว่ามี error ก้อนใหญ่ไหม) และ R^2 (สัดส่วนความแปรปรวนที่อธิบายได้)

ขอบเขต: ทำนายเฉพาะชั่วโมงที่ระบบเปิดให้บริการ (Functioning Day = Yes) · ระดับทั้งเมือง ไม่ใช่รายสถานี · ไม่ใช่ยอดเช่าของชั่วโมงก่อนหน้า (lag) เพราะเป็นการใช้ target ทำนาย target · โจทย์คือ "อากาศ + เวลาแบบนี้ → จะมีคนเช่ากี่คัน" ไม่ใช่การพยากรณ์อนุกรมเวลา

ทำไมเลือก regression: target ของชุดข้อมูลเป็นจำนวนนับต่อเนื่อง ทำนายได้ตรงๆ โดยไม่ต้องตั้ง threshold เอง และตัวเลข "กี่คัน" นำไปวางแผนได้ทันที

2. ข้อมูล

ชุดข้อมูล **Seoul Bike Sharing Demand** (UCI Machine Learning Repository, id 560, สัญญาอนุญาต CC BY 4.0) บันทึกยอดเช่ารายชั่วโมงของระบบจักรยานสาธารณะกรุงโซล ตั้งแต่ 1 ค.ศ. 2017 ถึง 30 พ.ย. 2018

ตารางที่ 2.1 ตัวแปรในชุดข้อมูล (เปลี่ยนชื่อเป็น snake_case)

คอลัมน์ดิบ	ชื่อที่ใช้	ชนิด / หน่วย
Date	date	วันที่ (dd/mm/yyyy) — ใช้เป็น group ของการ split ไม่ใช่ feature
Rented Bike Count	rented_bike_count	target · คำนวณชั่วโมง
Hour	hour	0–23
Temperature(°C)	temp_c	°C
Humidity(%)	humidity_pct	%
Wind speed (m/s)	wind_speed_ms	m/s
Visibility (10m)	visibility_10m	หน่วย 10 เมตร (27–2,000)
Dew point temperature(°C)	dew_point_c	°C
Solar Radiation (MJ/m2)	solar_radiation_mj	MJ/m ²
Rainfall(mm)	rainfall_mm	มม.
Snowfall (cm)	snowfall_cm	ซม.
Seasons	seasons	Winter / Spring / Summer / Autumn
Holiday	holiday	Holiday / No Holiday
Functioning Day	functioning_day	Yes / No (ระบบเปิด/ปิด)

การตรวจโครงสร้าง: ไฟล์ไม่ใช่ UTF-8 (สัญลักษณ์ ° เก็บเป็น byte 0xB0) จึงอ่านด้วย cp1252 และระบุรูปแบบวันที่ %d/%m/%Y เอง (ตรวจจากค่าส่วนแรกที่ขึ้นถึง 31) ได้ **8,760 แถว × 14 คอลัมน์** ครบ 365 วัน × 24 ชั่วโมง ไม่มี missing value และไม่มีแถวซ้ำ

3. การแบ่งข้อมูลและการป้องกัน leakage

ชั่วโมงติดกันในวันเดียวกันมีอากาศและยอดเช่าใกล้เคียงกันมาก ถ้าสุ่มแบ่งทีละแถว ชั่วโมง 14:00 อาจอยู่ใน train ส่วน 15:00 อยู่ใน test โมเดลจะแค่ "จำวันนั้น" ได้ ทำให้คะแนนสูงเกินจริง จึงใช้ **วันที่เป็นกลุ่ม (group)** ให้ทั้ง 24 ชั่วโมงของวันหนึ่งอยู่ฝั่งเดียวกันเสมอ

ตารางที่ 3.1 ผลการแบ่งด้วย GroupShuffleSplit (test_size = 0.2 ของจำนวนวัน, random_state = 42)

	วัน	แถว	ชั่วโมงที่ระบบปิด	วันหยุดนักขัตฤกษ์	สัดส่วนฤดูหนาว
train	292	7,008	240	12	22.9%
test	73	1,752	55	6	31.5%

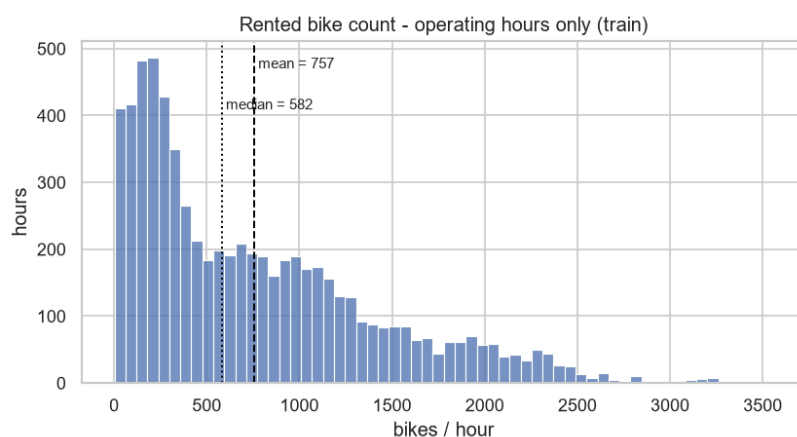
- เลือกสุ่มวันจากทั้งปี (แทนการตัดตามเวลา) เพราะข้อมูลมีแค่ 1 ปี ถ้าตัดตามเวลา test จะมีแค่ 1–2 ฤดูที่โมเดลไม่เคยเห็น
- test มีสัดส่วนฤดูหนาวสูงกว่า train จากความแปรปรวนของการสุ่มเพียง 73 วัน เราไม่สุ่มใหม่จนกว่าตัวเลขจะ "สวย" เพราะนั่นคือการเลือก test ด้วยมือ แต่บันทึกไว้เพื่อใช้ตีความผล test (หัวข้อ 11)
- รายการวันถูกบันทึกเป็นไฟล์ data/splits/train_dates.csv, test_dates.csv และ commit ไว้ทุก notebook ใช้วัน test ชุดเดียวกัน

- ทุกชั้นที่มี cross-validation ใช้ **5-fold GroupKFold** และส่ง groups = วันที่ทุกครั้ง เมื่อทดลองกับ KFold แบบสุ่มแถวพบว่า validation fold มีแถวจาก 281 วัน และ **ทุกวันมีชั่วโมงอื่นของวันเดียวกันอยู่ใน train** ส่วน GroupKFold มีวันซ้ำ 0 วันในทุก fold

4. การสำรวจข้อมูล (EDA) — ใช้ train เท่านั้น

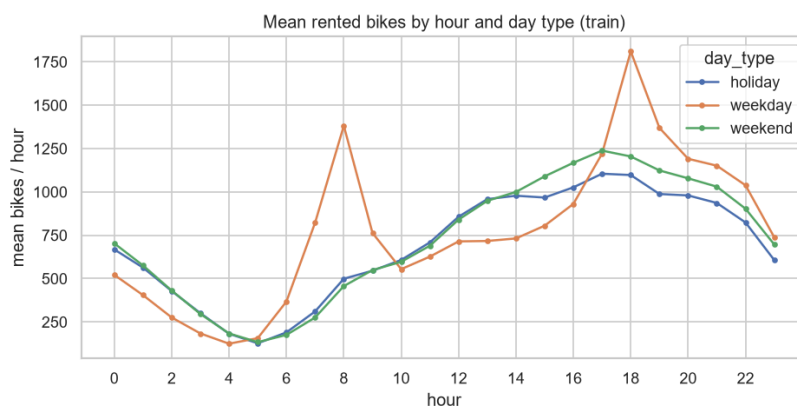
4.1 ชั่วโมงที่ระบบปิด และ distribution ของ target

ใน train มี 240 แถวที่ `functioning_day = No` (ระบบปิดทั้งวัน 10 วัน) ยอดเช่าเป็น 0 ทุกแถว และแถวที่ยอดเป็น 0 ทั้งหมดคือแถวเหล่านี้ ถ้าเก็บไว้ โมเดลจะได้คะแนนฟรีจากกฎ "ปิด → 0" ที่ผู้ให้บริการรู้อยู่แล้ว จึง **ตัดออกทั้ง train และ test** (เป็นกฎนิยามขอบเขต ไม่ได้เรียนค่าจากข้อมูล) หลังตัดเหลือ **6,768 แถว (282 วัน)** ยอดต่ำสุด 2 คัน mean 756.8 median 582.5 skewness 1.07 (เบ้ขวา)



รูปที่ 4.1 distribution ของยอดเช่ารายชั่วโมง (train, เฉพาะชั่วโมงที่ระบบเปิด)

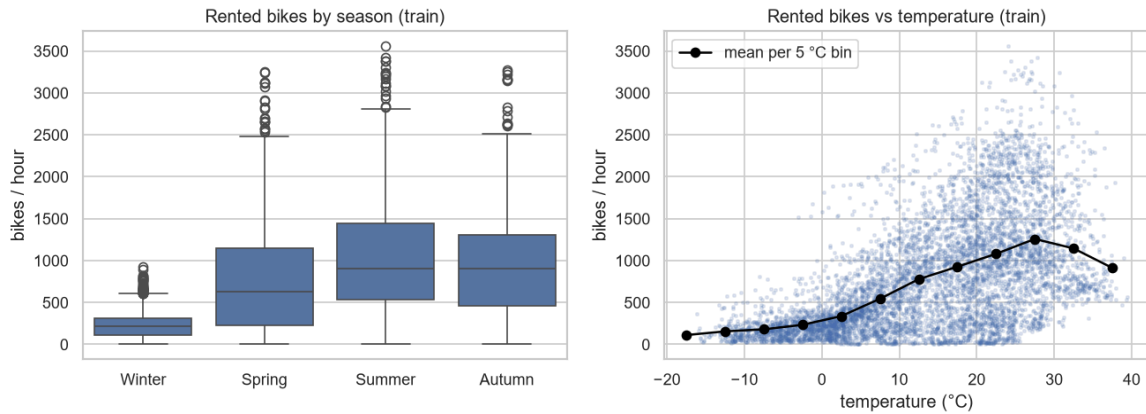
4.2 รูปแบบรายชั่วโมงและประเภทวัน



รูปที่ 4.2 ยอดเช่าเฉลี่ยรายชั่วโมง แยกวันทำงาน / เสาร์-อาทิตย์ / วันหยุดนักขัตฤกษ์

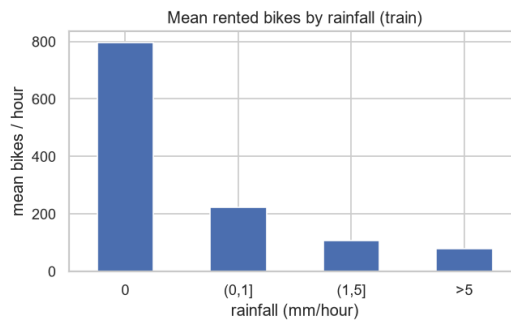
วันทำงานมีพีคแหลม 2 ครั้งที่ **8:00** (~1,380 คัน) และ **18:00** (~1,810 คัน) ซึ่งเป็นการเดินทางไป-กลับที่ทำงาน ส่วนเสาร์-อาทิตย์และวันหยุดนักขัตฤกษ์มีรูปร่างเดียวกัน คือไม่มีพีคเช้าและสูงสุดราว 17:00 ทุกประเภทวันต่ำสุดตอน 4:00–5:00 → ชั่วโมงสัมพันธ์กับยอดเช่าแบบ **ไม่เป็นเส้นตรงและเป็นวงรอบ** และรูปร่างขึ้นกับประเภทวัน (interaction)

4.3 ฤดู อุณหภูมิ และฝน



รูปที่ 4.3 ยอดเช่าตามฤดู (ซ้าย) และตามอุณหภูมิพร้อมค่าเฉลี่ยทุกช่วง 5 °C (ขวา)

- ฤดูหนาวเฉลี่ยเพียง ~230 คัน/ชม. เทียบกับ 774–1,037 ในฤดูอื่น
- ยอดเพิ่มตามอุณหภูมิจนสูงสุดที่ 25–30 °C (~1,258) แล้ว **ลดลง** เมื่อร้อนเกิน 30 °C → ความสัมพันธ์เป็นโค้งคว่ำ ซึ่งโมเดลเชิงเส้นจับไม่ได้
- ฝนเป็น 0 ใน 93.9% ของชั่วโมง แต่ชั่วโมงที่ฝนตกยอดตกจาก ~795 เหลือ ~164 คัน และแม้ฝนแค่ ≤ 1 มม. ยอดก็เหลือ ~223 → ผลของ **"มีฝนหรือไม่"** สำคัญกว่าปริมาณฝน



รูปที่ 4.4 ยอดเช่าเฉลี่ยตามปริมาณฝน

4.4 Correlation และ outlier

- มีคู่ตัวแปรเดียวที่ $|r| > 0.8$ คือ `temp_c` กับ `dew_point_c` ($r = 0.91$) · กับ target: `temp_c` $r = 0.55$, `hour` $r = 0.44$ · ฝน/หิมะมี r ต่ำ (-0.13 , -0.15) ทั้งที่มีผลแรง เพราะความสัมพันธ์ไม่เป็นเส้นตรง
- **ไม่ตัด outlier** ของสภาพอากาศและยอดเช่า เพราะเป็นเหตุการณ์จริง (ฝนหนัก, ลมแรง, ชั่วโมงพีค) ที่โมเดลต้องเจอตอนใช้งาน · กฎ IQR ใช้ไม่ได้กับตัวแปรที่ส่วนใหญ่เป็น 0 (ทุกชั่วโมงที่ฝนตกกลายเป็น "outlier")
- **ความชื้น = 0%** มี 17 แถว (19–28 พ.ค. 2018) เมื่อคำนวณย้อนจากอุณหภูมิและ dew point ด้วยสูตร Magnus ได้ 33–72% จึงเป็น **ค่าผิดของเซนเซอร์** → แปลงเป็น NaN แล้วเติมด้วยค่ามัธยฐานที่เรียนจาก training fold ใน Pipeline

5. Feature engineering และ preprocessing

ตารางที่ 5.1 feature ที่สร้างเพิ่ม (คำนวณจากแถวเดียวกันเท่านั้น ไม่ใช้ target)

feature	สร้างจาก	เหตุผล (จาก EDA)
hour_sin, hour_cos	sin/cos($2\pi \cdot \text{hour}/24$)	ชั่วโมงเป็นวงรอบ: 23:00 กับ 0:00 ห่างกัน 0.261 บนวงกลม เท่ากับ 0:00–1:00 (ตัวเลขติดห่าง 23)
is_weekend, day_of_week	date	รูปร่างรายชั่วโมงของวันหยุดต่างจากวันทำงาน
is_holiday	holiday	แปลงข้อความเป็น 0/1
month	date	ละเอียดกว่าฤดู (ต้น/ปลาย Spring ต่างกันมาก)
is_raining	rainfall_mm > 0	"มีฝน" ลดยอด ~80% ไม่ว่าจะตกมากหรือน้อย

Pipeline: ทุกขั้นที่เรียนค่าจากข้อมูลอยู่ใน Pipeline เพื่อให้ CV fit ใหม่ทุก fold — ตัวแปรตัวเลขและ binary ผ่าน SimpleImputer(median) → StandardScaler ส่วน seasons, month, day_of_week ผ่าน OneHotEncoder ตามด้วยช่อง select (สำหรับ selector/PCA) และช่อง model

การ **encode** ชั่วโมง เลือกได้ 3 แบบตามชนิดโมเดล เพราะ sin/cos คู่เดียวให้โมเดลเชิงเส้นสร้างฟังก์ชันได้แค่ 1 ครั้งต่อวัน แต่ข้อมูลมี 2 ฟังก์ชัน:

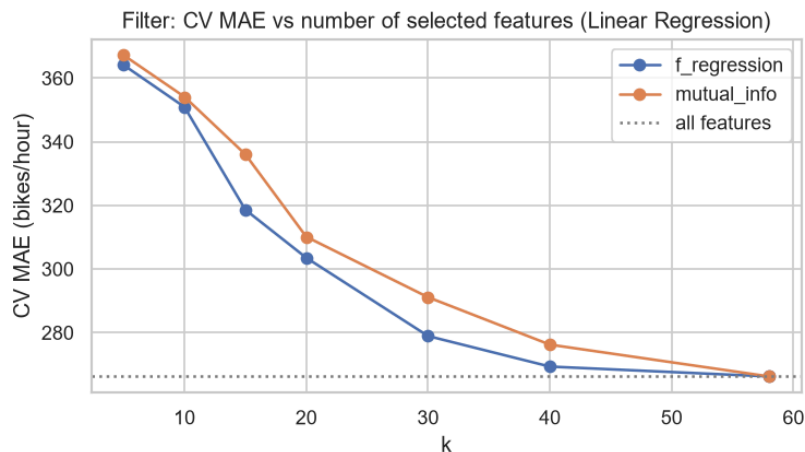
โมเดล	encoding	คอลัมน์หลัง preprocess	เหตุผล
Linear Regression	one-hot 24 คอลัมน์	58	ให้แต่ละชั่วโมงมีค่าคงที่ของตัวเอง รองรับ 2 ฟังก์ชัน
kNN	sin/cos	36	ใช้ระยะทาง ต้องการให้ 23:00 ใกล้ 0:00 และมีคติน้อย
Random Forest, HistGradientBoosting	0–23 ตรงๆ	35	ต้นไม้ตัดสินใจแบ่งช่วงชั่วโมงได้เองหลายครั้ง

6. Feature selection

ทำ 3 แนวทาง โดย selector อยู่ในช่อง select ของ Pipeline และเลือกจำนวน feature ด้วย GridSearchCV + GroupKFold · ประเมินด้วย Linear Regression (hour one-hot, 58 คอลัมน์) เพราะ feature selection มีผลกับโมเดลเชิงเส้นมากที่สุด หมายเหตุ: ใช้ **chi²** ไม่ได้ เพราะ chi² ต้องการ target เป็นหมวดหมู่ (และต้องการค่า feature ไม่ติดลบ ขณะที่อุณหภูมิติดลบได้) จึงใช้ f_regression และ mutual_info_regression แทน

ตารางที่ 6.1 สรุปผล feature selection (CV MAE, คั่น/ชม.)

วิธี	แนวทาง	พารามิเตอร์ที่ดีที่สุด	จำนวน feature	CV MAE
ไม่เลือก (ทุก feature)	—	—	58	266.2
SelectKBest(f_regression)	Filter	k = 58	58	266.2
SelectKBest(mutual_info)	Filter	k = 58	58	266.2
RFE (จัดอันดับด้วย Ridge)	Wrapper	n = 30	30	265.8
SelectFromModel(Lasso)	Embedded	alpha = 0.1	52	266.2



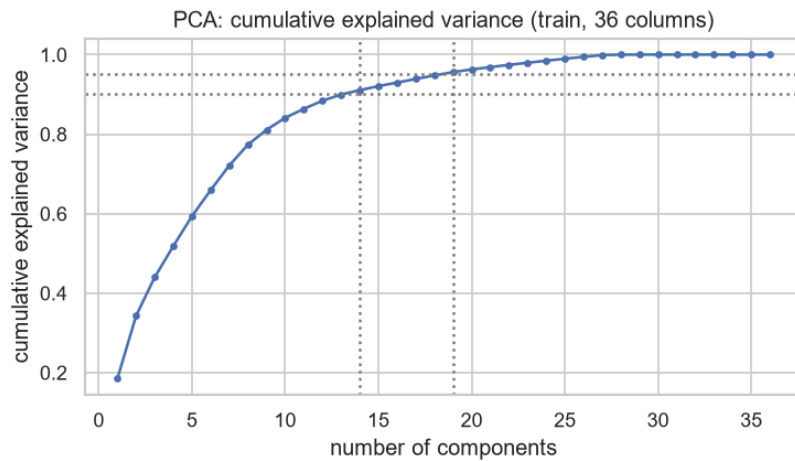
รูปที่ 6.1 CV MAE ตามจำนวน feature ที่ filter เลือก (k)

- **Filter:** MAE ลดลงเรื่อยๆ เมื่อ k เพิ่ม และดีที่สุดเมื่อใช้ทุกตัว เพราะ filter ให้คะแนนที่ละคอลัมน์ แต่ one-hot ของชั่วโมงมีประโยชน์ *เมื่ออยู่ด้วยกัน* · ทั้งสองวิธีจัด `temp_c` เป็นอันดับ 1 และ mutual information จับผลของฝน (`is_raining`, `rainfall_mm`) ได้ดีกว่า F-test
- **Wrapper (RFE):** ใช้ 30 จาก 58 feature ได้ MAE เท่าเดิม (ต่าง 0.4 คั่น น้อยกว่า std 4.8) — ตัดข้อมูลซ้ำซ้อนทิ้ง เช่น `is_weekend/is_holiday` กับ one-hot ของวัน, เดือนกับฤดู และ `rainfall_mm` (เก็บ `is_raining`)
- **Embedded (Lasso):** ดัน coefficient เป็น 0 ให้ one-hot ที่ซ้ำกับกลุ่มอื่น 6 ตัว (เช่น `seasons_Summer`, `month_7`) และให้ coefficient ใหญ่สุดกับชั่วโมงพีค (`hour_18` +837 คั่น) · ต่างจากที่คาดไว้ Lasso ไม่ได้ตัด `dew_point_c` ทั้งที่ r กับ `temp_c` สูง

ตัดสินใจ: ไม่มีวิธีไหนทำให้ MAE ดีขึ้นเกินระดับ noise จึงใช้ทุก feature ในโมเดลสุดท้าย และใช้ผลชั้นนี้อธิบายความสำคัญและความซ้ำซ้อนของ feature

7. PCA

ทำ PCA บนข้อมูลหลัง standardize (36 คอลัมน์, hour แบบ sin/cos) ต้องใช้ **14 components** จึงอธิบาย variance ได้ $\geq 90\%$ และ 19 components สำหรับ $\geq 95\%$ → ข้อมูลซ้ำซ้อนพอสมควรแต่บีบมิติได้ไม่มาก



รูปที่ 7.1 cumulative explained variance

ตารางที่ 7.1 การตีความ principal component แรกๆ จาก loading

PC	variance	loading หลัก	ตีความ
PC1	18.5%	humidity +0.53, dew point +0.39, visibility -0.32, is_raining +0.31	ความชื้น / อากาศมัว
PC2	15.8%	temp +0.56, dew point +0.45, solar +0.44, hour_cos -0.30	ความอุ่น / กลางวัน
PC3	9.7%	rainfall +0.51, is_raining +0.49, hour_cos -0.36	ฝน

ตารางที่ 7.2 CV MAE (ต้น/ชม.) เมื่อใส่ PCA ใน Pipeline เทียบกับไม่ใส่

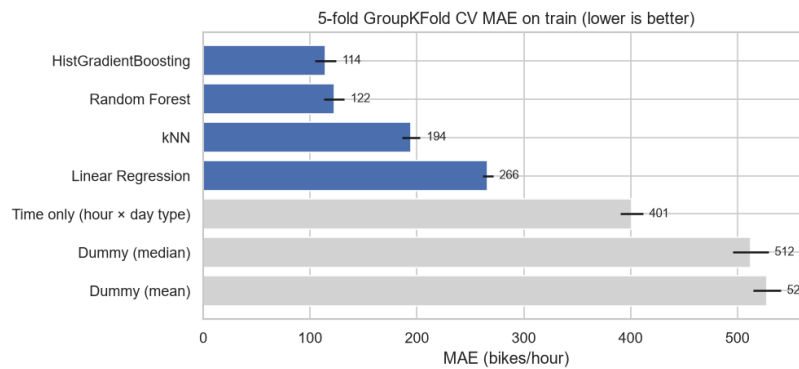
โมเดล	ไม่มี PCA	5 PC	10 PC	15 PC	20 PC	25 PC
Linear Regression	314.6	364.9	340.2	325.6	329.2	315.6
kNN (k = 10)	194.4	200.8	190.5	189.5	201.3	195.0

PCA ไม่ช่วย Linear Regression (ตัดข้อมูลบางส่วนทิ้ง ส่วนโมเดลเชิงเส้นบนแกนที่หมุนแล้วก็ยังเป็นเชิงเส้นเหมือนเดิม) และช่วย kNN เล็กน้อย (~5 คั่น) เพราะการตัดมิติที่เป็น noise ทำให้ "เพื่อนบ้าน" มีความหมายขึ้น → ใช้ PCA(15) เป็นตัวเลือกหนึ่งใน tuning ของ kNN

8. Baseline และการเทียบโมเดล

ตารางที่ 8.1 baseline และ 4 โมเดลด้วยค่า default (5-fold GroupKFold บน train, หน่วยต้น/ชม.)

โมเดล	CV MAE	± std	RMSE	R ²	MAE บน train	gap
Dummy (mean)	527.5	12.9	650.4	-0.003	527.1	—
Dummy (median)	512.2	16.6	673.6	-0.075	511.8	—
เวลาอย่างเดียว (hour × ประเภทวัน)	401.0	10.5	520.2	0.358	396.0	—
Linear Regression (hour one-hot)	266.2	4.8	347.7	0.713	259.8	6.4
kNN (k = 10, sin/cos)	194.4	8.6	289.9	0.800	151.9	42.5
Random Forest (200 ต้น)	122.4	9.4	199.4	0.904	36.8	85.6
HistGradientBoosting	114.1	10.0	184.6	0.918	69.0	45.1



รูปที่ 8.1 CV MAE $\pm$ std (สีเทา = baseline)

- baseline "เวลาอย่างเดียว" คือค่าเฉลี่ยของแต่ละ (ชั่วโมง $\times$ ประเภทวัน) ที่เรียนจาก training fold ใช้วัดว่า **ข้อมูลอากาศช่วยจริงไหม** — ทุกโมเดลชนะ baseline นี้ชัดเจน
- **Linear Regression** gap แค่ 6.4 คั่น = underfit (จับความสัมพันธ์โค้งและ interaction ไม่ได้) · **Random Forest** MAE บน train แค่ 36.8 และ gap 85.6 = overfit มากที่สุด (bias ต่ำ variance สูง) · **HistGradientBoosting** แม่นที่สุดและ gap ปานกลาง
- ลำดับ เส้นตรง $\rightarrow$ ระยะทาง $\rightarrow$ ต้นไม้ ตรงกับที่ EDA คาดไว้ (ความสัมพันธ์โค้ง + interaction ชั่วโมง $\times$ วัน)

9. Hyperparameter tuning

tune ด้วย `GridSearchCV` (Linear, kNN) และ `RandomizedSearchCV` (RF สุ่ม 15 จาก 64 ชุด, HGB สุ่ม 30 จาก 480 ชุด) ทั้งหมดใช้ `GroupKFold` + `groups` และเลือกด้วย MAE · ลองฝึกบน `log1p(y)` ผ่าน `TransformedTargetRegressor` ด้วย (แปลงค่าทำนายกลับก่อนคิดคะแนน MAE จึงยังเป็นหน่วยคั่น)

ตารางที่ 9.1 CV MAE ก่อนและหลัง tune (คั่น/ชม.)

โมเดล	ก่อน	หลัง	พารามิเตอร์ที่ได้
Linear Regression	266.2	266.2	ไม่มีไฮเปอร์พารามิเตอร์
Linear Regression + $\log1p(y)$	—	211.0 $\pm$ 6.9	ดีขึ้น 55 คั่น (-21%)
kNN	194.4	183.7 $\pm$ 7.8	$k = 5$, weights = distance, PCA(15)
kNN + $\log1p(y)$	—	188.7 $\pm$ 9.3	$k = 15$, distance
Random Forest	122.4	121.8 $\pm$ 9.1	200 ต้น, min_samples_leaf = 1, max_features = 0.75
HistGradientBoosting	114.1	109.0 $\pm$ 8.1	learning_rate = 0.02, max_iter = 400, max_leaf_nodes = 127, min_samples_leaf = 5, l2 = 1.0
HistGradientBoosting + $\log1p(y)$	—	110.7 $\pm$ 8.0	พารามิเตอร์เดียวกับ HGB

- **$\log1p$ ช่วยโมเดลเชิงเส้นมาก** เพราะผลของอากาศเป็นแบบ "คูณ" (ฝนตก $\rightarrow$ ยอดลด $\sim 80\%$ ไม่ว่ายอดปกติจะเป็นหลักร้อยหรือหลักพัน) บนสเกล log ผลแบบคูณกลายเป็นแบบบวกซึ่งโมเดลเชิงเส้นจับได้ · แต่ไม่ช่วย kNN และ HGB ซึ่งไม่ได้สมมติรูปร่างความสัมพันธ์อยู่แล้ว
- พารามิเตอร์ที่ดีที่สุดของ HGB อยู่ที่ชอบของ grid (learning_rate ต่ำสุด, max_leaf_nodes สูงสุด) ขยาย grid ได้อีก แต่ส่วนที่เหลือน่าจะน้อยกว่า std (~ 8 คั่น)

10. การเลือกโมเดลและทรัพยากรที่ใช้

ตารางที่ 10.1 เปรียบเทียบความแม่นยำและทรัพยากร (ตัวที่ดีที่สุดของแต่ละตระกูลหลัง tune)

โมเดล	CV MAE	fit ทั้ง train (s)	ไฟล์ joblib (MB)	latency / 1 แกว (ms)	ต้นทุนการทำนาย
HistGradientBoosting	109.0 ± 8.1	2.97	5.77	7.0	O(จำนวนต้น × ความลึก)
Random Forest	121.8 ± 9.1	3.20	121.72	9.4	O(จำนวนต้น × ความลึก) · ไฟล์ใหญ่
kNN (+PCA 15)	183.7 ± 7.8	0.02	1.81	2.7	O(n·d) ต่อครั้ง ต้องเก็บ train ทั้งหมด
Linear Regression + log1p(y)	211.0 ± 6.9	0.02	0.01	2.6	O(d)

เวลา fit และ latency เปลี่ยนเล็กน้อยทุกครั้งที่รันและขึ้นกับเครื่อง ใช้ดูลำดับขนาดเท่านั้น (latency = มัธยฐานของการทำนาย 1 แกว 30 ครั้ง รวม preprocessing ใน Pipeline)

เลือก HistGradientBoosting: MAE ต่ำที่สุด ห่างจาก Random Forest 12.8 คั่น (~1.5 เท่าของ std จึงไม่ถือว่าเสมอกัน) ไฟล์เพียง ~5.8 MB และทำนาย 1 แกวใช้ไม่กี่มิลลิวินาที · Random Forest ตกไปเพราะทั้งแม่นน้อยกว่าและไฟล์ ~122 MB เกินขีดจำกัด 100 MB ของ GitHub

11. ผลบนชุดทดสอบและ error analysis

นำ HistGradientBoosting ที่ fit บน train ทั้งหมดมาทำนาย test 1,697 แกว เป็นครั้งแรกและครั้งเดียว (ค่าทำนายติดลบ 3 แกว clip เป็น 0)

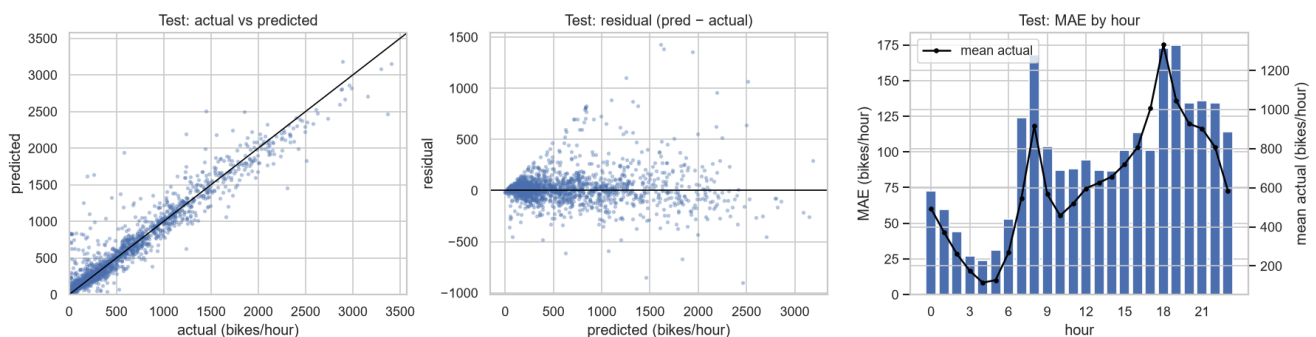
ตารางที่ 11.1 CV เทียบ test

เมตริก	CV บน train	test
MAE (คั่น/ชม.)	109.0	97.4
RMSE (คั่น/ชม.)	—	170.1
R ²	—	0.919

test MAE ใกล้กับ CV แปลว่าโมเดลไม่ได้ overfit และ CV ประเมินได้สมจริง ที่ test ต่ำกว่า CV ~12 คั่น สอดคล้องกับที่บันทึกไว้ในหัวข้อ 3: test มีชั่วโมงฤดูหนาวมากกว่า ซึ่งยอดต่ำ ค่าคลาดเป็นคั่นจึงเลือกตาม ไม่ได้แปลว่าโมเดลเก่งขึ้น

ตารางที่ 11.2 error แยกกลุ่ม (bias = ค่าเฉลี่ยของ ทำนาย - จริง, บวก = หายสูงเกิน)

กลุ่ม	n	ยอดเฉลี่ย	MAE	bias	MAE / ยอดเฉลี่ย
Winter	552	211.7	49.7	+5.5	23.5%
Spring	336	596.0	97.0	+30.3	16.3%
Autumn	425	805.5	126.4	+49.7	15.7%
Summer	384	1,018.1	134.3	+20.6	13.2%
วันทำงาน	1,128	681.3	102.6	+18.6	15.1%
วันหยุด	569	495.4	87.2	+37.4	17.6%
ฝนตก	103	180.2	159.8	+71.2	88.7%
ไม่มีฝน	1,594	647.3	93.4	+21.9	14.4%



รูปที่ 11.1 ค่าจริง vs ค่าทำนาย (ซ้าย), residual (กลาง), MAE รายชั่วโมงเทียบกับยอดเฉลี่ย (ขวา)

- ค่าทำนายเกาะเส้นทแยงดี แต่ **ความคลาดไต่ตามระดับยอด** (residual เป็นรูปกรวย) · MAE สูงสุดที่ชั่วโมงพีค 18:00–19:00 และ 8:00 ต่ำสุดที่ 3–5
- **ชั่วโมงฝนตกคือจุดอ่อนที่สุด**: คลาด 88.7% ของยอดและหายสูงเกิน (ชั่วโมงฝนตกมีแค่ ~6% ของ train ให้เรียน)
- bias เป็นบวกทุกกลุ่ม → โดยรวมโมเดลหายสูงกว่จริงเล็กน้อย
- **10 ชั่วโมงที่ผิดมากที่สุด 5 ชั่วโมงมาจากรันเดียว** คือ 23 ส.ค. 2018 เวลา 18:00–22:00 ซึ่งอากาศในข้อมูลดูปกติ แต่ยอดจริงต่ำมาก (เช่น 21:00 จริง 189 หาย 1,613) วันนั้นพายุไต้ฝุ่น Soulik ขึ้นฝั่งเกาหลีใต้ อีกกรณีคือ 24 ก.ย. 2018 ช่วงเทศกาลชูชอกที่คนเดินทางออกจากโซล → error ก้อนใหญ่มาจาก **เหตุการณ์ที่ไม่มีใน feature**

12. การทำนายข้อมูลใหม่

สร้างสถานการณ์สมมติในรูปแบบเดียวกับข้อมูลดิบ ผ่านฟังก์ชันสร้าง feature ชุดเดียวกับ train (dew point คำนวณจากอุณหภูมิและความชื้นให้สอดคล้องกันทางกายภาพ) แล้วทำนายด้วยโมเดลที่เลือก

ตารางที่ 12.1 ผลทำนายสถานการณ์สมมติ

สถานการณ์	ทำนาย (คัน/ชม.)	ความหมาย
พุธ 18:00 ฤดูร้อน 24 °C ไม่มีฝน	2,981	พีดเย็นวันทำงาน + อากาศดี = ต้องเตรียมจักรยานมากที่สุด
พุธ 18:00 เดียวกัน แต่ฝนตก 3 มม. (20 °C, ชื้น 95%)	204	ฝนทำให้ยอดขายไป ~93%
เสาร์ 18:00 อากาศเหมือนแถวแรก	2,167	วันหยุดไม่มีพีดเดินทางกลับบ้าน ต่ำกว่าวันพุธ ~800 คัน
พุธ 08:00 ฤดูหนาว -3 °C	690	ยังมีพีดเช้า แต่ต่ำกว่าฤดูร้อนมาก

ทิศทางของทุกปัจจัยสมเหตุสมผลและตรงกับ EDA แต่ค่าทำนายของชั่วโมงฝนตกมีความไม่แน่นอนสูงที่สุด (หัวข้อ 11)

13. ความท้าทายที่พบ

ปัญหา	วิธีแก้
ชั่วโมงในวันเดียวกันคล้ายกันมาก (leakage แบบซ่อน)	แบ่ง train/test และทำ CV ตามวันด้วย GroupShuffleSplit / GroupKFold และแสดงให้เห็นว่า KFold ธรรมดาทำให้ทุกวันใน validation รั่วไปอยู่ใน train
ชั่วโมงที่ระบบปิด ยอด = 0 เสมอ	ตัดออกและประกาศขอบเขตของโมเดล แทนที่จะปล่อยให้คะแนนขึ้นแบบไม่มีความหมาย
ไฟล์ไม่ใช่ UTF-8 และวันที่กำกวม	ตรวจ byte ดิบ เลือก cp1252 และระบุรูปแบบวันที่เอง
ความชื้น 0% (ค่าผิดของเซนเซอร์)	ยืนยันด้วยสูตร Magnus แล้วแปลงเป็น NaN + เติมค่าใน Pipeline
chi ² ใช้กับ regression ไม่ได้ (และต้องการค่าไม่ติดลบ)	ใช้ f_regression และ mutual_info_regression
one-hot ของเดือน/ฤดู/วันซ้อนกัน (multicollinearity)	ใช้ Ridge จัดอันดับใน RFE และ Lasso ตัด one-hot ที่ซ้ำ
ชั่วโมงมี 2 พีด แต่ sin/cos ให้โมเดลเชิงเส้นได้แค่ 1 พีด	เลือก encoding ของชั่วโมงตามชนิดโมเดล
Random Forest ไฟล์ ~122 MB	ใช้ขนาดไฟล์เป็นเกณฑ์หนึ่งในการเลือกโมเดล

14. ข้อจำกัดและแนวทางพัฒนาต่อ

- ชั่วโมงที่ฝนตก คลาดมาก → เพิ่มข้อมูลหลายปี หรือถ่วงน้ำหนักชั่วโมงฝนตกตอนฝឹก
- เหตุการณ์พิเศษ (พายุ, เทศกาล, ประกาศเตือนภัย) ไม่อยู่ใน feature → เพิ่ม feature ปฏิทินเทศกาลหรือค่าเดือนสภาพอากาศ
- ข้อมูลมีแค่ 1 ปีและเป็นยอดรวมทั้งเมือง ใช้วางแผนรายสถานีไม่ได้ และใช้ได้เฉพาะชั่วโมงที่ระบบเปิด
- split แบบสุ่มวันวัดความสามารถ "อากาศ + เวลา → ยอดเช่า" ไม่ได้วัดการพยากรณ์ล่วงหน้าหลายเดือน
- input ตอนใช้งานจริงคือ *พยากรณ์* อากาศ ซึ่งคลาดเคลื่อนมากกว่าค่าที่วัดได้จริงในชุดข้อมูล → ความแม่นยำจริงน่าจะต่ำกว่าที่รายงาน
- พารามิเตอร์ของ HGB อยู่ที่ชอบ grid ขยายการค้นหาได้อีก

15. สรุป

HistGradientBoostingRegressor ทำนายยอดเช่าจักรยานรายชั่วโมงได้คลาดเคลื่อนราว **100 คัน/ชม.** (CV 109.0, test 97.4) อธิบายความแปรปรวนได้ ~92% และลดความคลาดเคลื่อนจากการใช้แค่ปฏิทินได้ ~73% แสดงว่าข้อมูลอากาศมีประโยชน์จริง ปัจจัยที่สำคัญที่สุดคือชั่วโมง (พีคเดินทาง 8:00 และ 18:00 ในวันทำงาน) อุณหภูมิ (ความสัมพันธ์โค้ง) และการมีฝน ผลนี้ได้มาภายใต้การป้องกัน leakage อย่างเข้มงวด (split และ CV ตามวัน, ทุกชั้นที่เรียนจากข้อมูลอยู่ใน Pipeline, เปิด test ครั้งเดียว) จึงเชื่อถือว่าสะท้อนการใช้งานกับวันที่โมเดลไม่เคยเห็น

ภาคผนวก ก. เช็คลิสต์ป้องกัน leakage

ข้อ	สถานะ
split train/test ตามวัน และทุก CV ใช้ GroupKFold + ส่ง groups	ผ่าน
ไม่มี feature ที่คำนวณจาก target (ไม่มี lag หรือค่าเฉลี่ยย้อนเช่า)	ผ่าน
rented_bike_count ไม่อยู่ใน X	ผ่าน
imputer, scaler, encoder, selector, PCA และการแปลง log1p อยู่ใน Pipeline	ผ่าน
การตัดแถวระบบปิดและการแปลงความชื้น 0% เป็นกฎตายตัว ไม่ได้เรียนค่าจากข้อมูล	ผ่าน
ไม่ได้เปิด test ระหว่าง EDA จนถึงเลือกโมเดล และประเมิน test ครั้งเดียว	ผ่าน

ภาคผนวก ข. การรันซ้ำ

โค้ดทั้งหมดอยู่ใน notebook 3 ไฟล์ รันตามลำดับ 00_setup → 01_eda → 02_modeling บน Python 3.13 ด้วยไลบรารีตาม requirements.txt (scikit-learn 1.9.1, pandas 2.3.3) · ทุกจุดที่มีการสุ่มใช้ random_state = 42 · เมื่อรันซ้ำบนเครื่องใหม่ได้ MAE, RMSE, R² และพารามิเตอร์ที่ดีที่สุดตรงกันทุกค่า ต่างกันแค่เวลา fit/latency · โมเดลสุดท้ายบันทึกที่ models/model.joblib (สร้างใหม่ได้จาก 02_modeling ชั้น 12) · รายงานนี้สร้างจาก reports/report.html ด้วย python reports/build_pdf.py

เอกสารอ้างอิง

- Seoul Bike Sharing Demand [Dataset]. (2020). UCI Machine Learning Repository. <https://doi.org/10.24432/C5F62R> (CC BY 4.0)
- Sathishkumar V E, Park J., Cho Y. (2020). Using data mining techniques for bike sharing demand prediction in metropolitan city. *Computer Communications*, 153, 353–366.
- Pedregosa F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Korea Times (2018-08-24). Typhoon Soulik weakens after storming southern areas. <https://www.koreatimes.co.kr/southkorea/society/20180824/typhoon-soulik-weakens-after-storming-southern-areas>
- Wikipedia. Chuseok. <https://en.wikipedia.org/wiki/Chuseok>